AMR Project Report

Rahanuma Tabassum, Md. Taijul Islam

***Abstract*—This work presents the design and implementation of an autonomous mobile robot developed within the Robot Operating System 2 (ROS 2) framework for mapping, localization, navigation, and exploration in partially known environments. The architecture is organized around four core milestones - Occupancy grid mapping via Simultaneous Localization and Mapping (SLAM), Global path planning using A^* paired with a Potential field planner for real-time local obstacle avoidance, robust pose estimation utilizing Monte Carlo Localization (MCL), and autonomous exploration driven by a frontier-based algorithm.Critical operational challenges—including sensor noise,localization drift, real-time obstacle avoidance, and efficient path planning. The combined pipeline allows the platform to achieve autonomous navigation in dynamic environment alongside continuous map refinement.This document covers the system design, debugging challenges, and practical solutions used to build a robust, self-navigating robot.The complete project code is available on GitHub: https://github.com/taijulhbrs/amr_team_ttp**

***Index Terms*—ROS2, Particle Filter, Monte Carlo Localization, A* Algorithm, Potential Field, Frontier Exploration, Robile, Autonomous Navigation**

I. Introduction

Autonomous mobile robots play a critical role in logistics, search-and-rescue, industrial automation, and exploration.Navigating unknown environments, avoiding obstacles, and exploring new areas are key to achieving full autonomy.This project develops an autonomous navigation system for the Robile platform using ROS2, integrating SLAM for mapping, A* for global path planning, the Potential Field method for obstacle avoidance, Monte Carlo Localization for accurate positioning, and frontier-based exploration for complete map coverage.Divided into four main milestones—mapping, navigation, localization, and exploration—this project demonstrates the robot's ability to operate autonomously in real-world environments. This report details the methodology, analyzes key technical challenges and their solutions, and highlights practical insights gained while developing a dependable autonomous navigation system.

II. Related Work

A. Localization

Monte Carlo Localization (MCL), first introduced by Dellaert et al. [1] , relies on particle filters to determine a robot's coordinates. we implemented Monte Carlo Localization (MCL), a particle filter-based approach, using real-time sensor data from LiDAR and odometry to estimate the robot's pose. MCL is widely used in probabilistic robotics as it maintains multiple hypotheses about the robot's location, refining them as new observations are received.

B. Path Planning

Path and motion planning are essential components of autonomous navigation, enabling a mobile robot to move efficiently towards a goal while avoiding obstacles. the A* algorithm is deployed as the global planner over discrete grid representations due to its optimality and efficiency. It combines actual movement cost with a heuristic estimate to find the shortest path. For local planning, the potential field method [2] provides real-time obstacle avoidance by combining attractive forces toward the goal and repulsive forces away from obstacles. we implemented a hybrid planning system combining A* global planner with potential field local planner to ensure smooth and dynamic navigation.

C. Exploration

Frontier-based exploration was introduced by Yamauchi [3] as a way for mobile robots to explore unknown areas by heading toward the borders between mapped and unmapped space. Even though this method has been expanded in many directions over the years, it is still one of the most reliable approaches for autonomous mapping. In our project, we implemented this idea on the Robile platform. The system reads the occupancy grid map from the SLAM module, detects frontier cells along the boundary, and automatically assigns them as navigation goals.

III. Methodology

A. Mapping

Mapping is a fundamental step in autonomous navigation, as it provides the robot with a structured representation of its surroundings, allowing for effective path planning and obstacle avoidance. Throughout this project, we used the Robile 4 platform and implemented SLAM (Simultaneous Localization and Mapping) using the robile navigation package. The mapping process was conducted following the official Robile-AMR documentation [4] , ensuring a structured and reproducible workflow. Initially,we opted for a simplified environment with only one obstacle, resulting in a clearer and more precise map.This technique helped limit spatial distortion, yielding a cleaner occupancy grid for stable robot navigation. The mapping process involved launching the robot with:

```
ros2 launch robile_bringup robot.launch.py
```

and starting the mapping node using:

```
ros2 launch robile_navigation online_async. launch.py
```

Next,we attempted to map an environment containing multiple obstacles to test the robot's ability to detect and navigate around multiple objects. The final map was saved using:

```
ros2 run nav2_map_server map_saver_cli -f ~/maps /map_02
```

B. Localization

The main focus of Part 2 was implementing Monte Carlo Localization (MCL) using a particle filter. Localization is essential for autonomous navigation because it allows the robot to track its position inside a previously mapped area. In this project, we implemented Monte Carlo Localization (MCL) from scratch to estimate the robot's pose on a known map using real-time wheel odometry and LiDAR scans . Because MCL is a probabilistic method, that maintains multiple hypotheses about the robot's location, refining them as new observations are received. The localization process involves four main steps: particle generation, motion update, measurement update, and resampling.

1) Particle Initialization: At the start of localization, we initialized the robot's position using an estimated pose from RViz. This was achieved through the 2D Pose Estimate tool, where an initial hypothesis about the robot's position was set manually. Once the initial pose was provided, the particle filter algorithm generated multiple particles around this location, representing possible poses of the robot. Each particle was associated with a weight, reflecting the likelihood of the robot being at that position. The set of particles was initialized as a Gaussian distribution centered around the estimated pose to provide a reasonable starting point.

2) Motion Model: The motion model updates particles based on odometry differences. Each particle's pose is then updated as:

$$x' = x + \hat{\delta}_{\text{trans}} \cos(\theta + \hat{\delta}_{\text{rot1}}) \tag{1}$$

$$y' = y + \hat{\delta}_{\text{trans}} \sin(\theta + \hat{\delta}_{\text{rot1}}) \tag{2}$$

$$\theta' = (\theta + \hat{\delta}_{\text{rot1}} + \hat{\delta}_{\text{rot2}}) \tag{3}$$

Where the planar movement is decomposed into an initial rotation , a translation , and a final rotation , with stochastic Gaussian noise added via motion parameters to model wheel slippage. This differential update prevents cumulative error, enabling the particle cloud to reliably track the robot's trajectory.

3) Measurement Model: The measurement update step refined the particle weights using LiDAR sensor readings. Each particle simulated a laser scan based on its pose and compared it with the actual scan received from the robot's LiDAR. Particles whose simulated scans closely matched the real sensor readings were assigned higher weights, whereas those that significantly deviated were given lower weights. This step effectively discarded incorrect hypotheses and retained the most probable locations.

4) Resampling: We implemented low-variance resampling to maintain particle diversity while concentrating on high-weight hypotheses. This method ensures that particles with higher weights are more likely to be retained.

5) Robot Model Visibility: Our implementation followed the structure of MCL-based localization in ROS2. The MonteCarloLocalizer node subscribed to multiple topics: /initialpose for setting the robot's initial pose, /odom for receiving odometry data, /scan for laser scan updates, and /map for loading the static environment map.Following iterative Bayesian updates, the node computes the weighted mean state and continually disseminates the estimated pose. It broadcasts a dynamic map $\rightarrow$ odom transform over the /tf tree—replacing external localization nodes such as AMCL—to maintain continuous coordinate consistency without disturbing the high-frequency odometry pipeline. Simultaneously, the particle distribution is published to /particle_cloud for RViz visualization, while the finalized robot pose estimate is output to /pf_pose for downstream navigation and path-planning modules.

One of the major challenges in real-time Monte Carlo Localization is managing the heavy computational load from high-density LiDAR scans across hundreds of particles. Evaluating every beam causes significant processing latency, making the filter lag behind the robot's physical movement. To overcome this, we implemented a laser subsampling approach that selects every 10th reading; because consecutive beams are spatially redundant, this cuts computation by nearly 90% while maintaining accurate convergence. Another key issue is the inefficiency and noise sensitivity of traditional ray-casting, which slows down the update step and fails under slight sensor inaccuracies. We addressed this by adopting an endpoint model paired with a precomputed likelihood field. Instead of tracing full rays, the algorithm only computes the obstacle hit points and performs quick distance lookups, making the localization both computationally lightweight and resilient to sensor noise.

C. PATH AND MOTION PLANNING

For Part 1, we implemented a hybrid planning system combining A* global planner with potential field local planner.

1) Global Planner (A):* The global planner subscribes to the occupancy grid map (/map), which provides a representation of the environment, indicating free spaces, obstacles, and unknown areas, from the robot's start pose to goal pose (/goalpose) topics. The A* algorithm is used to compute the shortest path by evaluating possible paths based on a cost function. This function consists of the actual cost of movement from the start to a given node and a heuristic estimate of the cost to reach the goal. The grid-based representation of the environment allows A* to find an efficient path while ensuring collision avoidance.Instead of sending every single grid cell, the system extracts only the key turning points where the path changes direction. This simplified route is then published on /global_path, which significantly reduces data load and helps the robot follow the path much more smoothly.

The A* algorithm operates by searching for the lowestcost path through a grid representation of the environment. It initializes the starting node and expands the search by evaluating neighboring nodes based on their cost. The cost function consists of two components:

$$f(n) = g(n) + h(n) \tag{4}$$

where g(n) represents the actual cost from the start to the current node and h(n) is a heuristic estimating the cost from the current node to the goal. The Euclidean distance

was chosen as the heuristic function to efficiently guide the search towards the goal. As the search progresses, the path is reconstructed from the goal back to the start, ensuring an optimal trajectory. However, raw paths generated by A* often contain unnecessary waypoints, which were removed using a path simplification technique that filters out collinear points.

2) Local Planner (Potential Field Method): The local planner uses attractive forces to pull the robot toward the goal and repulsive forces to push it away from obstacles.
The attractive force is:

$$\mathbf{F}_{\text{att}} = k_{\text{att}} \cdot (\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{robot}}) \tag{5}$$

The repulsive force is:

$$F_{\text{rep}} = -k_{\text{rep}} \sum_i \left(\frac{1}{r_i} - \frac{1}{r_{\max}} \right) \frac{1}{r_i^2} \cdot \hat{r}_i \tag{6}$$

where r is the distance to the nearest obstacle. This ensures that the robot repels strongly from nearby objects while maintaining a safe path toward the goal.

During testing, we encountered a major challenge with our artificial potential field planner: the robot suffered from erratic behavior and violent oscillations near obstacles, which frequently disrupted path tracking.In narrow corridors, competing repulsive forces from opposing walls caused the potential field planner to stall and oscillate or rotate aggressively in place instead of moving forward. To solve this, we recalibrated the controller around physical velocity limits and metric units.We resolved this by fine-tuning the attractive and repulsive gains ($k_{att} = 1.0, k_{rep} = 1.5$) alongside tightening the obstacle influence radius, balancing the repulsive vectors to maintain a dominant forward heading through confined spaces.

3) Integration: The integration of global and local planners significantly improved the robot's navigation performance. The global planner ensured an optimized high-level trajectory, while the local planner provided dynamic adjustments to avoid obstacles. This allowed the robot to navigate large environments while avoiding obstacles in real-time.

D. Environment Exploration

For Part 3, we implemented frontier-based exploration. The exploration node subscribes to the map, identifies frontier cells (free cells adjacent to unknown cells), and publishes them as red markers for visualization. It then selects a frontier as the next goal and publishes it to /goal_pose, which the navigation system executes.

1) Frontier-Based Exploration : The robot's exploration strategy relied on the occupancy grid map generated by SLAM, which classified the environment into three types of regions: free space (0), occupied space (1), and unknown space (-1). The algorithm identified frontiers by scanning for free space cells that bordered unknown areas. These frontier cells served as potential exploration goals. Once a frontier was identified, the robot used the A* global path planner to determine the optimal path from its current position to the selected frontier goal while avoiding large obstacles. The computed path was then followed using the local potential field planner, which dynamically adjusted the robot's movements based on real-time obstacle detection. If the A* planner determined that a goal was unreachable due to obstacles, the system automatically discarded the goal and generated a new frontier point. This ensured continuous exploration without requiring manual intervention.

2) Implementation in ROS2: The implementation of the exploration system was based on ROS2 and involved multiple components interacting through topic subscriptions and publications. The key elements included:

a) Occupancy Grid Map Subscription: The robot subscribed to the /map topic to receive real-time updates of the environment. The received occupancy grid was processed into a 2D array to classify free space, obstacles, and unexplored areas. This served as the foundation for frontier detection.

b) Frontier Detection: The algorithm scanned the occupancy grid for free-space cells adjacent to unknown areas, marking these as frontier points. The function _find_frontier_clusters() iterated through the occupancy grid and identified potential exploration targets.

c) Goal Selection and Publishing: Once frontier clusters were identified, their centroid was converted from grid coordinates to real-world coordinates. The closest frontier relative to the robot's current pose was then published as a PoseStamped message under the topic /goal_pose.

d) Goal Validation: Rather than relying on explicit planner feedback, the system monitors target reachability through spatial tolerance and temporal limits. If the robot reaches within the goal tolerance (0.4 m) or fails to reach the target within a designated timeout (25.0 s due to unreachability or obstruction), the current goal is cleared. This triggers the algorithm to discard the stalled target and select a new frontier centroid, ensuring continuous exploration.

e) Navigation Execution: Once a valid goal was selected, the robot followed the computed path using the local potential field planner. The local planner adjusted the robot's movement in real time to avoid obstacles while steering toward the target.

Initially, frontiers were not showing in RViz. We discovered this was a QoS issue—the map subscription needed TRANSIENT_LOCAL durability. After fixing this, red dots appeared at the edges of the map, and the robot autonomously moved to explore them, expanding the map progressively.Additionally, during implementation, in environments with complex layouts, the A* planner occasionally failed to find a valid path due to tight spaces. To improve navigation smoothness, motion parameters in the potential field planner were fine-tuned to prevent excessive oscillations in the robot's trajectory.

IV. Evaluation

The implemented system was tested on the real Robile robot in a laboratory environment. Each component was evaluated independently and then as an integrated system.

A. Mapping Results

Initially, we opted for a simplified environment with only one obstacle (Fig. 1), resulting in a clearer and more precise map.This technique helped limit spatial distortion, yielding a cleaner occupancy grid for stable robot navigation.

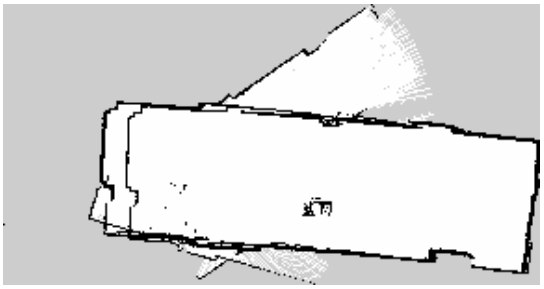


Fig. 1: Generated map with one obstacle (Map 01)

Next,we also attempted to map an environment containing multiple obstacles to test the robot's ability to detect and navigate around multiple objects (Fig. 2).



Fig. 2: Generated map with multiple obstacles (Map 02)

B. Localization Results

The particle filter successfully tracked the robot's pose in real-time. Figure 3 shows the red particle cloud clustered around the robot's actual position.

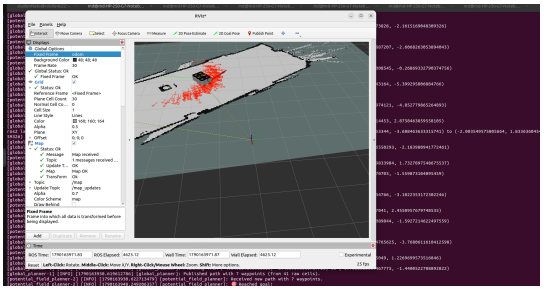


Fig. 3: Particle filter visualization: Red arrows represent particle hypotheses clustering around the robot

Key observations:

- Particles mostly remained clustered during forward/backward motion and rotations.
- The transform from map to odom was published correctly, enabling robot model visualization.
- The filter recovered quickly after manual displacement.

C. Path Planning Results

The hybrid A*/potential field planner was tested with single and multiple obstacles. Figure 4 shows the A* path generated from start to goal.

Key observations:

- A* generated optimal paths avoiding obstacles.
- Robot reached goals within 0.25m tolerance.

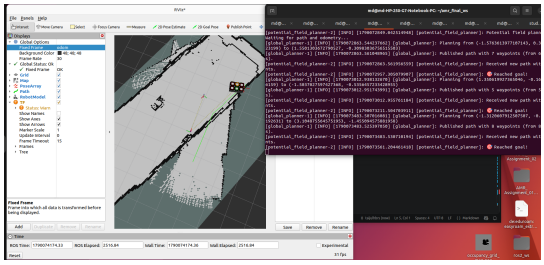


Fig. 4: Path followed by the robot

D. Exploration Results

The frontier-based exploration was tested in an unknown environment. Figure 5 shows frontiers (gray region) detected at the boundaries of explored space.

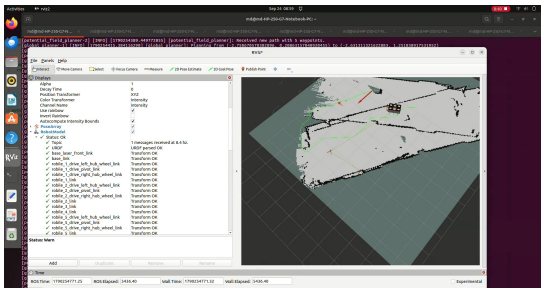


Fig. 5: Frontier detection: gray region indicate unexplored areas; robot autonomously navigates to them

Key observations:

- Frontiers were accurately identified at map boundaries.
- Robot autonomously moved to explore new areas.
- Map expanded progressively as exploration continued.

E. Challenges and Solutions

Throughout the project, several challenges were encountered that required debugging and tuning. The table below summarizes these challenges and the solutions implemented to resolve them.

TABLE I: Challenges and Solutions

Challenges	Solutions
Map not loading in RViz	fixed by activating map_server before opening RViz, or re-adding the display
Path planner using raw odometry instead of the particle filter's corrected pose	fixed by integrating TF lookups
Potential field planner oscillating/rotating in place in narrow corridors	fixed via gain tuning
Robot model not appearing	Added transform broadcaster from map to odom
Obstacle avoidance weak	Increased repulsive gain
Build errors	Fixed A* syntax errors and setup.py

F. Video Demonstrations

The recorded videos of the project are available at the following links:

- Mapping and Path Planning: https://www.youtube.com/watch?v=aU0tOj548yA [5]
- Localization: https://www.youtube.com/watch?v=9gbQaa3KoBE [6]
- Exploration: https://www.youtube.com/watch?v=JczIQYeqD-8 [7]

V. Conclusions

This project successfully implemented an autonomous mobile robot using ROS2 for mapping, navigation, localization, and exploration. The robot built an occupancy grid map, navigated using A* for global path planning and the Potential Field method for local obstacle avoidance, localized itself with Monte Carlo Localization, and explored unknown areas using frontier-based exploration. Challenges such as sensor noise, localization drift, and path optimization were addressed through tuning and algorithmic improvements. The integration of these techniques allowed the robot to adapt dynamically and update its understanding of the environment. This project demonstrates how autonomous robots can efficiently map and explore unknown spaces, contributing to advancements in robotic navigation.

A. Summary

- Localization: A particle filter with 300 particles accurately tracked the robot's pose within the map. The filter demonstrated robustness against sensor noise and odometry drift.
- Path Planning: The A* global planner computed optimal paths, while the potential field local planner handled realtime obstacle avoidance. Path simplification improved navigation smoothness.
- Exploration: Frontier-based exploration enabled the robot to autonomously identify and navigate to unexplored areas, achieving complete map coverage.

B. Contributions

- Rahanuma Tabassum: Mapping,Potential field and A* optimization,Exploration implementation, optimization optimization,debugging, testing, project report, and video editing.
- Md. Taijul Islam: Mapping, Potential field and A* implementation, Localization implementation, Exploration optimization, debugging, and testing.

C. Future Work

Potential improvements for future development include:

- Implementing adaptive particle resampling based on localization confidence.
- Adding dynamic obstacle detection and tracking.
- Optimizing frontier selection using clustering algorithms.
- Extending exploration to 3D environments
- Integrating Kalman filtering for odometry correction

References

[1] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte Carlo Localization for Mobile Robots," in *Proceedings 1999 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 1999, pp. 1322–1328.

[2] R. Labbe, "Kalman and Bayesian Filters in Python," https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/12-Particle-Filters.ipynb, 2026, accessed: 2026-03-21.

[3] B. Yamauchi, "A Frontier-Based Approach for Autonomous Exploration," in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA'97)*. IEEE, 1997, pp. 146–151.

[4] Robile-AMR, "Demo mapping," 2026, pp. 1–8, accessed: 2026-03-21.

[5] R. Tabassum, "Path Planning and Mapping," https://www.youtube.com/watch?v=aU0tOj548yA, Sep. 2026, youTube video, accessed: Sep. 26, 2026.

[6] ——, "Localization," https://www.youtube.com/watch?v=9gbQaa3KoBE, Sep. 2026, youTube video, accessed: Sep. 26, 2026.

[7] ——, "Exploration," https://www.youtube.com/watch?v=JczIQYeqD-8, Sep. 2026, youTube video, accessed: Sep. 26, 2026.

ACKNOWLEDGMENT

We would like to thank Prof. Javad Ghofrani for his guidance and support throughout this project. We also thank the Robile team for providing the robot platform and documentation. Special thanks to our peers for their feedback during the development and testing phases.